
Exercises: Advanced Lesson 9

Your Name: Mikey Ferguson

Your Math Class: MATH 7013 - Advanced Mathematical Modeling

1: Define a function: $f(x, y, z) = \{x, y, z\}$ such that argument: y has a default value of 10, and argument: z has a default value of 20. Evaluate: $f[1,2,3,4]$, $f[1,2,3]$, $f[1,2]$, $f[1]$, $f[]$

```
Clear[f, x, y, z];
f[x_, y_ : 10, z_ : 20] := {x, y, z}
?f

{f[1, 2, 3, 4], f[1, 2, 3], f[1, 2], f[1], f[]}
```

2: Make 2 copies of the following code segment by interchanging the second and third lines. Evaluate each of the 2 code segments. Also display `?f` for each of the 2 code segments. Observe the differences between the 2 code segments (and also differences between how you typed them in and how they are stored) and identify which code segment (first, second or both) that would work if we want the numbers to be identified as: even or odd or = 2.

```
f[x_] := "x is anything"
f[x_?OddQ] := "x is odd"
f[x_?EvenQ] := "x is even"
f[2] := "x = 2"
Table[{i, f[i]}, {i, 1, 4, 1/2}] // TableForm

Clear[f, i];
f[x_] := "x is anything";
f[x_?OddQ] := "x is odd";
f[x_?EvenQ] := "x is even";
f[2] := "x = 2"
?f

TableForm[Table[{i, f[i]}, {i, 1, 4,  $\frac{1}{2}}$ }]

Clear[f, i];
f[x_?OddQ] := "x is odd";
f[x_] := "x is anything";
f[x_?EvenQ] := "x is even";
f[2] := "x = 2"
?f
```

```
TableForm[Table[{i, f[i]}, {i, 1, 4,  $\frac{1}{2}}$ }]
```

```
Clear[f, i];
f[2] := "x = 2";
f[x_?OddQ] := "x is odd";
f[x_?EvenQ] := "x is even";
f[x_] := "x is anything"
```

```
TableForm[Table[{i, f[i]}, {i, 1, 4,  $\frac{1}{2}}$ }]
```

Both orders give the same result. Mathematica does not store definitions in the order they are typed: it puts more specific patterns first, so `f[2]` moves to the top and the catch-all `f[x_]` to the bottom whichever way they are entered. `?f` shows the same stored order both times and the two tables are identical, reading odd, anything, = 2, anything, odd, anything, even. Both segments therefore work for identifying even, odd and 2.

3: Make 2 copies of the following code segment by interchanging the second and third lines. Evaluate each of the 2 code segments. Also display `?f` for each of the 2 code segments. Observe the differences between the 2 code segments (and also differences between how you typed them in and how they are stored) and identify why you get different results.

```
f[x_?OddQ] := "x is odd"
f[x_?Positive] := "x is positive"
f[x_?IntegerQ] := "x is an integer"
f[2] := "x = 2"
f[x_] := "x is arbitrary"
Table[{i, f[i]}, {i, -2, 4, 1/2}] // TableForm
```

```
Clear[f, i];
f[x_?OddQ] := "x is odd";
f[x_?Positive] := "x is positive";
f[x_?IntegerQ] := "x is an integer";
f[2] := "x = 2";
f[x_] := "x is arbitrary"
?f
```

```
TableForm[Table[{i, f[i]}, {i, -2, 4,  $\frac{1}{2}}$ }]
```

```
Clear[f, i];
f[x_?OddQ] := "x is odd";
f[x_?IntegerQ] := "x is an integer";
f[x_?Positive] := "x is positive";
f[2] := "x = 2";
f[x_] := "x is arbitrary"
```

?f

```
TableForm[Table[{i, f[i]}, {i, -2, 4,  $\frac{1}{2}}$ }]
```

Here the order does matter. OddQ, Positive and IntegerQ are all the same shape of pattern, so Mathematica cannot rank them and keeps them in the order typed. Whichever comes first wins for a value that satisfies more than one test. At $x = 4$ the first version answers positive and the second answers integer.

4: Assume that in your research you construct a complicated new function called: myFunction [x]. Your work requires repeated integration involving your new function. However, *Mathematica* is unaware of your new function and its corresponding integral. Therefore you integrate your function to get: myFunctionIntegral [x]. You would like to assign your new formula to the *Mathematica* command Integrate by:

```
Integrate [ myFunction[x_], x_ ] := myFunctionIntegral [x]
```

Part 1: Try this and explain what happens.

Part 2: Tag this assignment to the functional argument: myFunction

Part 3: Evaluate: ?? myFunction

Part 4: Evaluate: $\int \text{Sin}[x] dx$

Part 5: Evaluate: $\int \text{myFunction}[x] dx$

Part 6: Evaluate: $\int \text{myFunction}[y] dy$

Part 7: What is the Head of: $\int \text{myFunction}[x] dx$

```
ClearAll[myFunction, myFunctionIntegral, x, y];
```

```
 $\int \text{myFunction}[x] dx := \text{myFunctionIntegral}[x]$ 
```

```
ClearAll[myFunction];
```

```
myFunction /:  $\int \text{myFunction}[x] dx := \text{myFunctionIntegral}[x];$ 
```

```
??myFunction
```

```
 $\int \text{Sin}[x] dx$ 
```

```
 $\int \text{myFunction}[x] dx$ 
```

```
 $\int \text{myFunction}[y] dy$ 
```

```
Head[ $\int \text{myFunction}[x] dx$ ]
```

Part 1: the direct assignment fails, because Integrate is Protected and Mathematica refuses to add a definition to it. Tagging the rule to myFunction instead stores it on your own symbol, which is allowed, and ??myFunction shows it there. After that the integral of myFunction[x] gives myFunctionIntegral[x], and it works for any variable name. Its Head is myFunctionIntegral.

5: Assume that in your research you construct a complicated new function called: myFunction [x, y, z, ...]. The order in which the functional arguments appear is irrelevant and the function should always

give the same value irrespective of the order of the arguments.

Part 1: Set your function's attributes to be orderless.

Part 2: Evaluate: ??myFunction

Part 3: Evaluate: myFunction[1,2,3] and myFunction[3,2,1]

Part 4: Evaluate: myFunction[a,b,c,d] and myFunction[d,c,b,a]

Part 5: Evaluate: myFunction[c,b,a,d] == myFunction[d,c,b,a]

```
ClearAll[myFunction, a, b, c, d];
```

```
SetAttributes[myFunction, Orderless];
```

```
??myFunction
```

```
{myFunction[1, 2, 3], myFunction[3, 2, 1]}
```

```
{myFunction[a, b, c, d], myFunction[d, c, b, a]}
```

```
myFunction[c, b, a, d] == myFunction[d, c, b, a]
```

6: Assume that in your research you construct a complicated new function called: myFunction [x, y, z, ...]. When you use your function, the first argument "x" is expected to contain information which will take on several different values while your function is being evaluated. However, all of the remaining arguments: y, z, ... will only be assigned one value each which will remain fixed throughout the evaluation of your function.

Part 1: Set your function's attributes to hold only the first argument.

Part 2: Evaluate: ?? myFunction

Part 3: Evaluate: a=1, b=2, c=3, d=4 and myFunction[a,b,c,d] Explain this result.

Part 4: Evaluate: myFunction[a_,b_,c_,d_] := {a, b, c, d}

Part 5: Evaluate: ?? myFunction

Part 6: Evaluate: a=1, b=2, c=3, d=4, myFunction[a,b,c,d]

Part 7: Explain (i.e. trace) at what point the argument "a" was evaluated.

```
ClearAll[myFunction, a, b, c, d];
```

```
SetAttributes[myFunction, HoldFirst];
```

```
??myFunction
```

```
a = 1;
```

```
b = 2;
```

```
c = 3;
```

```
d = 4;
```

```
myFunction[a, b, c, d]
```

```
myFunction[a_, b_, c_, d_] := {a, b, c, d}
```

```
??myFunction
```

```

a = 1;
b = 2;
c = 3;
d = 4;
myFunction[a, b, c, d]

```

```
Trace[myFunction[a, b, c, d]]
```

Part 3: HoldFirst holds only the first argument, so b, c and d evaluate to 2, 3 and 4 while a stays as the symbol a, giving myFunction[a, 2, 3, 4]. Part 7: the trace shows that a is not evaluated until the definition fires and a is substituted into the right-hand side, one step after the other three arguments have already been replaced by their values.

7: Assume that in your research you construct a complicated new function called: myFunction [x, y, z, ...]. It is important to document your function so that other users can understand its usage.

Part 1: Set your function's usage message to: "myFunction [x, y, z, ...] is important to understand."

```
Set: myFunction [x_, y_, z_] := Sin[x y z]
```

Set your function's attributes to hold all of the arguments.

Part 2: Evaluate: ? myFunction

Part 3: Evaluate: ?? myFunction

```

ClearAll[myFunction, x, y, z];
myFunction::usage = "myFunction [x, y, z, ... ] is important to understand.";
myFunction[x_, y_, z_] := Sin[x y z];
SetAttributes[myFunction, HoldAll];

?myFunction

??myFunction

```